

Module 4

UART — UVM+SV Verification

Learning objectives

- UART UVM agent: Transaction (UartTxTransaction), sequence (UartTxSequence), driver (UartTxDriver),...
- Interface: uart_tx_if (clk, rst_n, start, data, tx, baud_tick) connects UVM to DUT.
- Loopback check: TX path (driver → DUT → monitor → scoreboard) and RX path (DUT rx_valid/rx_data → s...

Prerequisites

- Module 1 (spec → RTL), Module 2 (UVM+SV, uvm_smoke), Module 3 (UART protocol + RTL + basic TB).
- Verilator, Make, C++ compiler, UVM (UVM_HOME or vendored UVM).

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["UART UVM Agent"]

T2["Loopback and RX Check"]

T1 --> T2

T3["Toolchain"]

T2 --> T3

Extend UART verification to ****UVM+SV**** — UART agent (transaction, sequence, driver, monitor, scoreboard); run on Verilator.

Overview

- Module 4 builds on Module 3 (UART protocol + RTL + basic testbench):

Syllabus topics

1. UART UVM Agent

- Transaction (UartTxTransaction): byte to send (data); monitor fills observed_tx (reconstructed from...
- Sequence (UartTxSequence): produces directed transactions (0x00, 0x01, 0x55, 0xAA, 0xFF).
- Driver (UartTxDriver): gets transaction, drives start and data for one cycle, waits 10 baud_ticks (...)
- Monitor (UartTxMonitor): watches tx line for start bit, samples 8 data bits per baud_tick, writes t...
- Scoreboard (UartTxScoreboard): compares expected (from test) vs observed_tx (from monitor); also ch...

2. Loopback and RX Check

- Loopback: TX output (vif.tx) is connected to RX input (dut_rx.rx). Bytes sent by driver are receive...
- RX hook: An initial block in the top looks up the scoreboard and calls `check_rx_byte(rx_data)` when...

3. Toolchain

- Same as Module 2: Verilator with -sv, --timing, --trace, UVM include paths, make SIM=verilator TEST...

Hands-on examples

Module 4 self-check

```
$ ./scripts/module4.sh --check
```

Terminal: module4_check

```
[[0;36m===== [[0m
[[0;36mModule 4: Demo commands (UART UVM) [[0m
[[0;36m===== [[0m

From repo root:

# 1) Environment sanity:
verilator --version
make --version
echo "$UVM_HOME"

# 2) Run the UART UVM example (inside the example directory):
cd module4/examples/uart_uvm
make SIM=verilator TEST=test_uart_uvm

# 3) Inspect logs and waveforms:
ls obj_dir
# If a VCD is produced:
ls *.vcd

[[1;33m[INFO] [[0m See module4/EXAMPLES.md and module4/examples/uart_uvm/README.md for more.
```

Example 1: UART UVM

- UVM phases and reporting (DRIVER, MONITOR, SCOREBOARD).
- TX and RX scoreboard checks (TX: monitor observes serial line; RX: loopback byte checked via scoreb...
- Final scoreboard summary (TX/RX matches and mismatches).

Demo: UART UVM

```
$ ./scripts/module4.sh --run
```

Terminal: ex01_uart_uvm

What to look for

- UVM phases and DRIVER, MONITOR, SCOREBOARD messages
- TX and RX scoreboard checks (TX: monitor observes serial line; RX: hook from DUT rx_valid/rx_data)

Practice & assessment

Exercises

- Run `uart_uvm`
- Trace one transaction
- Change the sequence
- Optional: Loopback path

Assessment checklist

- Can describe the UART UVM agent: transaction (one byte), sequence, driver, monitor, scoreboard.
- Can explain loopback and how TX and RX are both checked.
- Can run `uart_uvm` and interpret UVM output (phases, scoreboard summary).
- Ready for Module 5: SPI protocol + RTL + basic testbench.

Summary & next steps

- Extend UART verification to ****UVM+SV**** — UART agent (transaction, sequence, driver, monitor, scoreb...
- Complete module4/CHECKLIST.md
- Run `./scripts/module4.sh --check`
- Next: SPI